University of Canberra
Faculty of Science and Technology

Software Technology 2
(7170)
Week 6 Tutorial & Computer Lab
Assignment 1: Individual Project
Part 3: Linear Data Structures (Linked Lists, Stacks, Queues and Priority Queues)

- This lab activity is assessed and contributes to the Assignment 1 (Individual Project) mark.
- Please go through Week 6 lecture recordings and Week_6_Tutorial_and Computer_Laboratory.doc available after Module 6 in Canvas (Assignment 1/Individual Project preparation activities)
- For assessment of this activity, you need to complete the following in week 6 tutorial/lab:

  1) attend your allocated computer lab class in Week 6,

  2) complete all tasks listed in this handout,

  3) demonstrate the working activities to your tutor in class, and

  4) Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 6** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.

- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

Tasks

- Understanding **Linear Data Structures** (linked lists, stacks, queues and priority queues and their implementation in Python (**data_structures_practice.py**)
- **Benchmarking Performance** of Linear Data Structures (data_structures_practice.py)
- Prior Reading: **Week_6_Lecture** available in Module 6 in Canvas**.**

Download the **Week_6_Lab_Activities.zip** after Module 6 in Canvas, and unzip the folder. This folder content contains all the necessary activities needed for each task completion in this week.

Task 0: Background (Not assessed) – Refer Week 6 Lecture in Canvas
Understanding Linear Data Structures (Linked Lists, Stacks, Queues and Priority Queues)

- Why data structures matter ?
- Data Structures
  - Linear (Linked Lists, Stacks, Queues, Priority Queues …)
  - Nonlinear (Trees, Graphs …)
- Linked Lists, Stacks, Queues, Priority Queues
- Comparison and use cases

- **Why Data Structures?**
  - Efficient data organization and retrieval

- Impact on performance & memory
- Real-world examples: Browsers, OS, task schedulers, algorithms

---

- **What is a Linked List?**
  - Collection of nodes
  - Single linked list and Double linked list
  - Each node contains data and pointer to next
  - Can be singly or doubly linked
  - Dynamic size – no fixed size limit

- **Linked List - Advantages & Disadvantages**

Advantages	**Disadvantages**
Dynamic memory utilization	No random access (O(n) traversal)
Easy insertion and deletion	More memory usage due to pointers
Ideal for unknown or changing sizes	Comparatively slower than arrays

- **What is a Stack?**
  - LIFO (Last In First Out) data structure
  - Only top element accessible
  - Operations: push (insert), pop (remove), peek (view top)

- **Stack Use Cases**
  - Undo functionality
  - Syntax parsing
  - Back button in browsers
  - Expression evaluation

- **What is a Queue?**
  - FIFO (First In First Out) data structure
  - Insert at rear, remove from front
  - Operations: enqueue, dequeue, peek

- **Queue Use Cases**
  - Task scheduling
  - Data buffers
  - Handling requests

- **What is a Priority Queue?**
  - Like a queue but elements have priorities
  - Element with highest priority served first
  - Typically implemented with heaps

- **Priority Queue Use Cases**
  - CPU scheduling

- Dijkstra's shortest path algorithm
- Event-driven simulation

- **Comparison Table**

Data Structure	Access Time	Insertion Time	Deletion Time	Use Cases
Linked List	O(n)	O(1)/O(n)	O(1)/O(n)	Dynamic size lists, undo logs
Stack	O(1) (top)	O(1)	O(1)	Expression evaluation, call stacks
Queue	O(1) (front)	O(1)	O(1)	Task scheduling, buffering
Priority Queue	O(log n)	O(log n)	O(log n)	Task prioritization, graphs

---

1. **When to Use Which?**
   - Use **Linked List** for dynamic memory, easy inserts/deletes at head or tail
   - Use **Stack** for LIFO needs (undo, parsing)
   - Use **Queue** for FIFO processing (printing jobs, requests)
   - Use **Priority Queue** for tasks with importance levels
   - Some Python data structure tools: lists, deque, heapq
   - Choose appropriate DS based on use case & performance needs

---

2. **Task 1: Linked Lists, Stacks, Queues, and Priority Queues**

(a) Run the script **linked_lists_practice.py** for testing the simple python lists, single linked and double linked list data structure code and benchmark their performance for different data sizes. Report on the performance (screenshots)for each data size. Currently it is set for 1000. Test it for 10000 and 50000.

```
Testing with 10000 elements, search and delete value: 392462
Python list insert at beginning: 0.000008 s
Python list search: 0.000045 s
Python list delete: 0.000100 s
Singly Linked List insert at beginning: 0.000002 s
Singly Linked List search: 0.000152 s
Singly Linked List delete: 0.000306 s
Doubly Linked List insert at beginning: 0.000003 s
Doubly Linked List search: 0.000170 s
Doubly Linked List delete: 0.000305 s
```

```
Testing with 50000 elements, search and delete value: 714500
Python list insert at beginning: 0.000041 s
Python list search: 0.000271 s
Python list delete: 0.000671 s
Singly Linked List insert at beginning: 0.000003 s
Singly Linked List search: 0.000807 s
Singly Linked List delete: 0.001592 s
Doubly Linked List insert at beginning: 0.000005 s
Doubly Linked List search: 0.000867 s
Doubly Linked List delete: 0.001552 s
```

My benchmark results for 10,000 and 50,000 elements show that Singly and Doubly Linked Lists are the fastest for adding data at the very beginning. They maintained a consistent speed of about 0.000003s to 0.000005s, confirming that linked structures are highly efficient for head insertions since they only need to update a pointer instead of moving existing data. However, the Python list was significantly faster for searching and deleting. At 50,000 elements, it completed the search in 0.000271s, nearly three times faster than the Singly Linked List. This is because Python lists store data in one continuous block, which is much easier for the CPU to process than the "pointer chasing" required to find scattered linked list nodes.

(b) Run the script **stacks_queues_practice.py** for testing the stack, queue and priority queue datastructure code and double linked list code and comparing their performance for different data sizes. Report on the performance (screenshots) for each data size. Currently it is set for 1000. Test it for 10000 and 100000.

```
Benchmarking with 10000 values
Stack (list) Push: 0.000202 s
Stack (list) Peek: 0.000000 s
Stack (list) Pop: 0.000342 s
Queue (deque) Enqueue: 0.000268 s
Queue (deque) Peek: 0.000000 s
Queue (deque) Dequeue: 0.000366 s
Priority Queue (heapq) Push: 0.000870 s
Priority Queue (heapq) Peek: 0.000000 s
Priority Queue (heapq) Pop: 0.004018 s
```

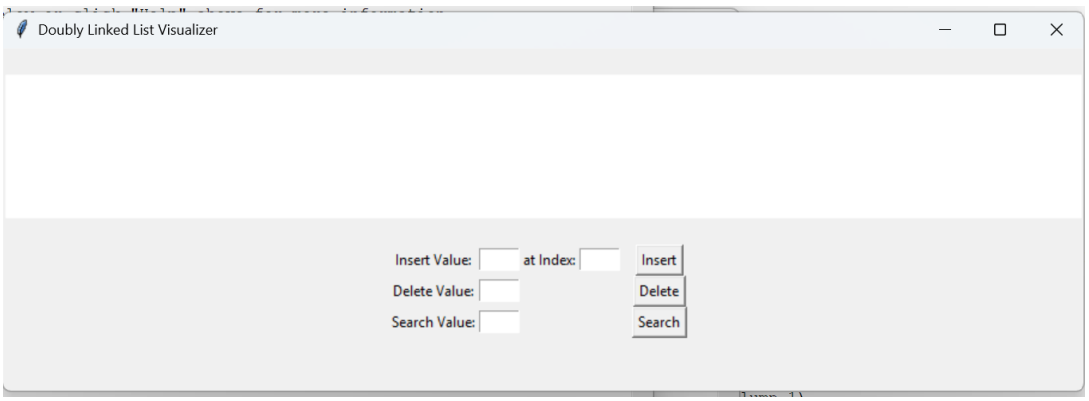
```
Benchmarking with 50000 values
Stack (list) Push: 0.001104 s
Stack (list) Peek: 0.000001 s
Stack (list) Pop: 0.001669 s
Queue (deque) Enqueue: 0.001431 s
Queue (deque) Peek: 0.000001 s
Queue (deque) Dequeue: 0.001984 s
Priority Queue (heapq) Push: 0.004960 s
Priority Queue (heapq) Peek: 0.000001 s
Priority Queue (heapq) Pop: 0.025565 s
```

My benchmark results for 10,000 and 50,000 values show that Stacks and Queues perform very similarly, with "Peek" operations being nearly instantaneous at 0.000001s. Pushing and popping for these structures scaled linearly, staying under 0.002s even at the higher data size. This confirms that for standard LIFO and FIFO tasks, both structures are highly efficient and scale predictably as data grows.

The Priority Queue, however, was significantly slower, especially for pop operations. At 50,000 values, popping from the Priority Queue took 0.025565s, which is over 12 times slower than a standard Queue. This performance hit happens because the heap must re-sort itself every time the highest-priority element is removed to maintain its structure.

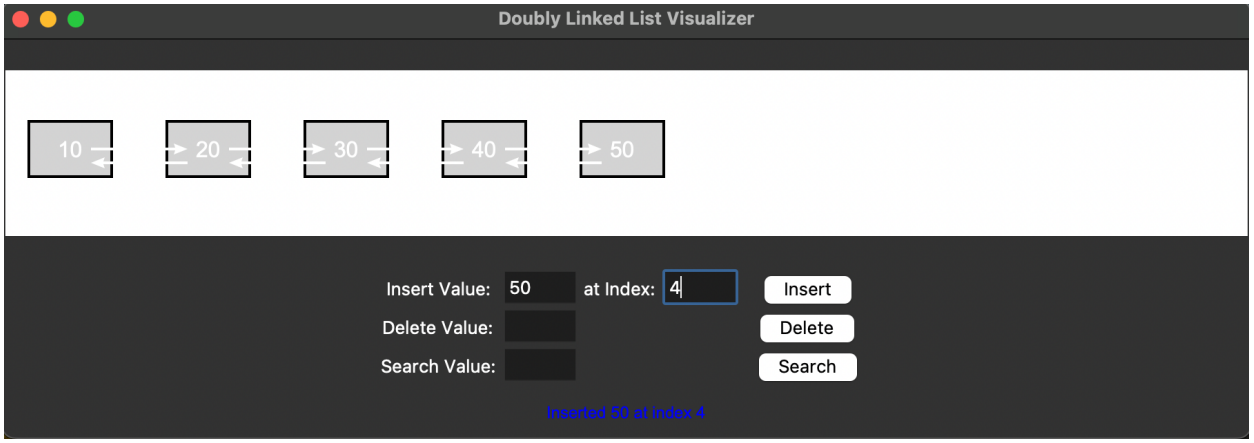
Task 2: Visualising Linked Lists

1. The python script **double_linked_list_visualiser.py** allows visualisation of search, insertion and delection in a double linked list (dll). A successful test execution of this script should display the following GUI interface:

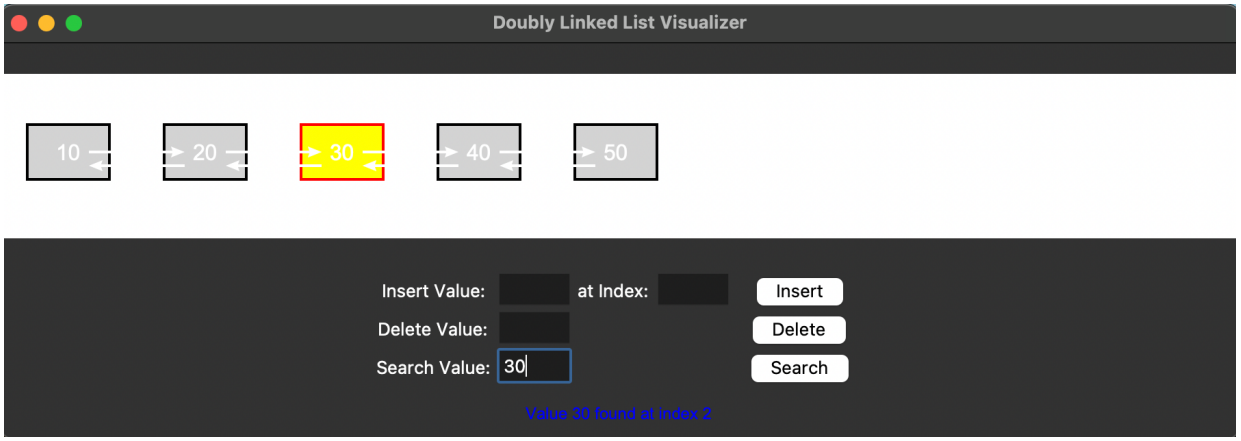


2. Test the code for inserting a list of integers (10,20,30, 40.50), and report the results (screenshots) for visualising insertion, search and deletion operations.

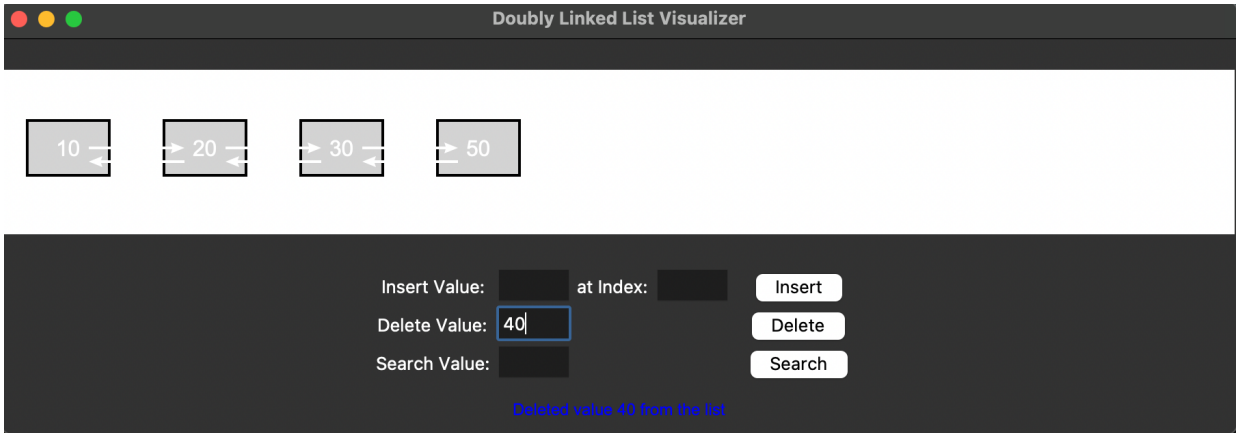
   - Test results (screenshots) when the **Insert** button inserts the value into the specified index in the dll list.



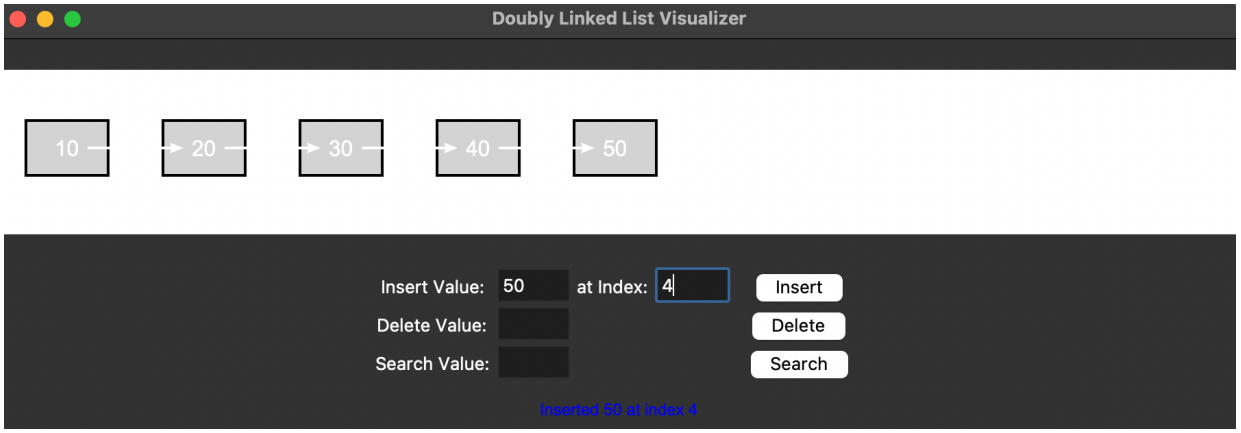
   - Test results when the **Search** button searches whether the specified value is in the dll list.



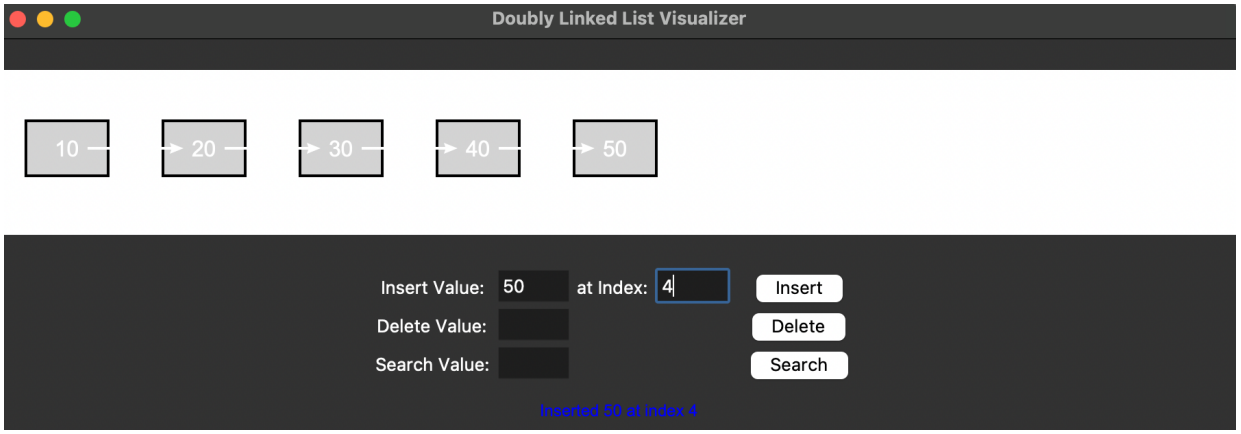
   - Test results when the **Delete** button deletes the specified value from the dll list.



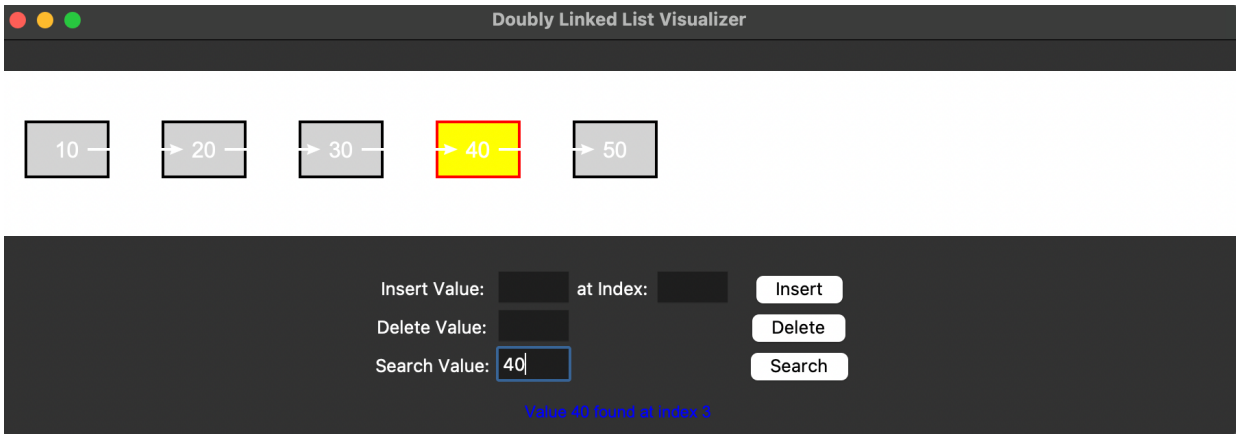
- **Modify the visualiser code** for this task to allow visualisation of **single linked list**. You can use the relevant classes from Task 1 related to single linked lists to write this single linked list visualiser.
- Show the GUI layout of a successful test run for this script.



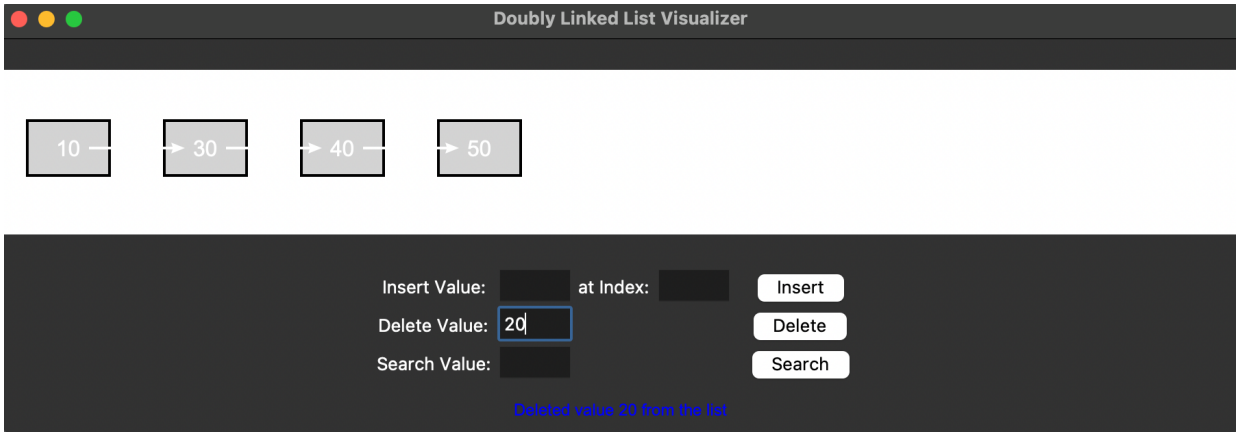
- Submit the code as **single_linked_list_visualiser.py**
- Show the results (screenshots) for insert, search and delete button operations (same test data - list of integers (10,20,30, 40.50))
- Test results when the **Insert** button inserts the value into the specified index in the sll list.



- Test results when the **Search** button searches whether the specified value is in the sll list.

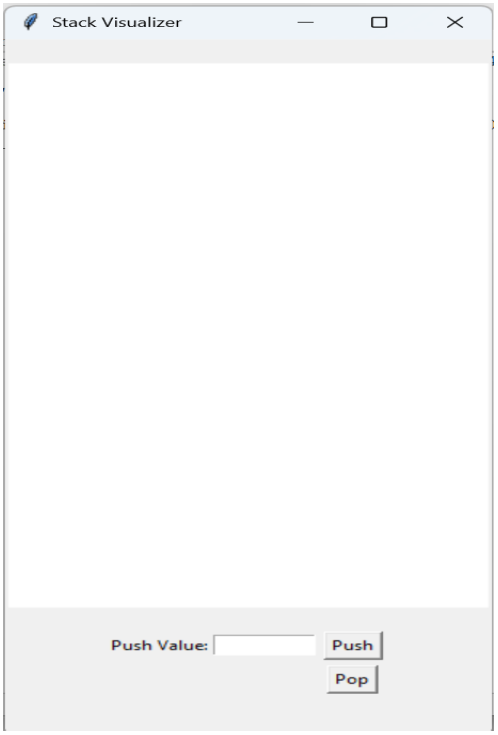


- Test results when the **Delete** button deletes the specified value from the sll list.



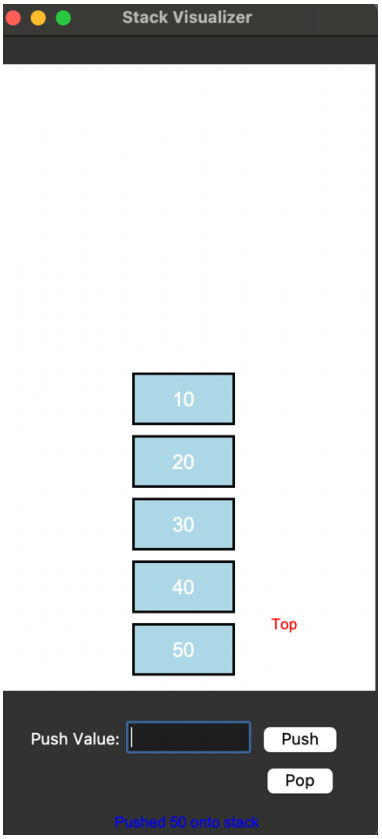
Task 3: Visualising Stacks and Queues

1. The python script **stack_visualiser.py** allows visualisation of LIFO push and pop operations for stack data structure. A successful test run of the code show display the following:

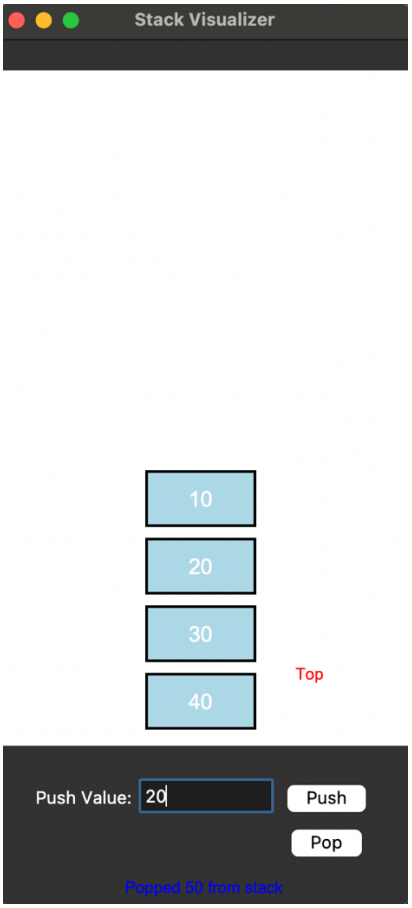


Test the visualiser for pushing a list of integers [10,20,30,40,50] and popping the list. Show the results (screenshots) of how the push and pop operations for a stack are performed in a LIFO order.

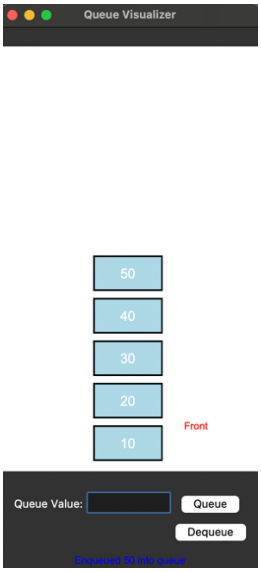
- Test results when the **Push** button inserts the value in the stack



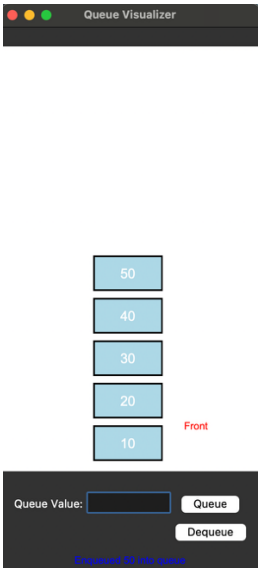
- Test results when the **Pop** button retrieves value from the stack.



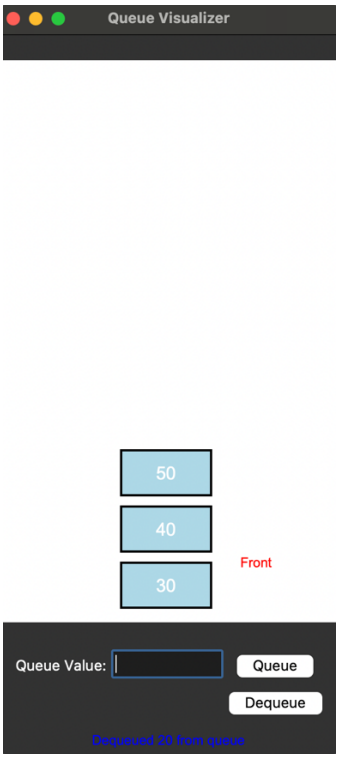
- **Modify the visualiser code** for this task to allow visualisation of **queue** . You can use the relevant classes from Task 2 related to queues to write this queue visualiser.
- Show the GUI layout of a successful test run for this script.



- Submit the code as **queue_visualiser.py**
- Show the results (screenshots) for push and pop button operations (same test data - list of integers (10,20,30, 40.50))
- Test results when the **push** button inserts the value into the queue.

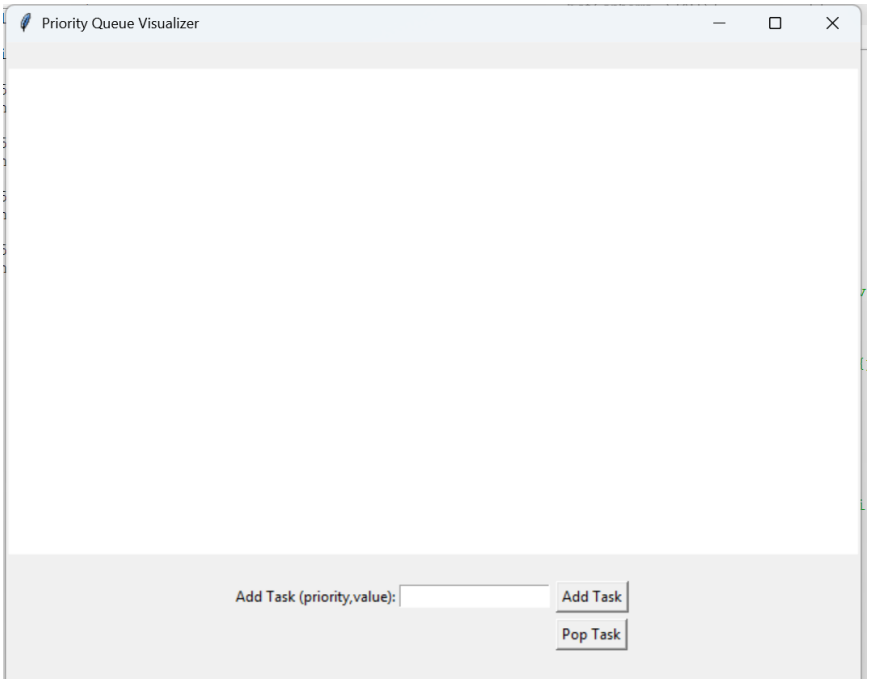


- Test results when the **pop** button retrieves the specified value from the queue.



Task 4: Visualising Priority Queues

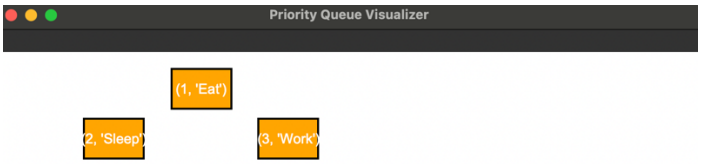
1. The python script **priority_queue_visualiser.py** allows visualisation of FIFO push and pop operations for priority queue data structure. A successful test run of the code show display the following:



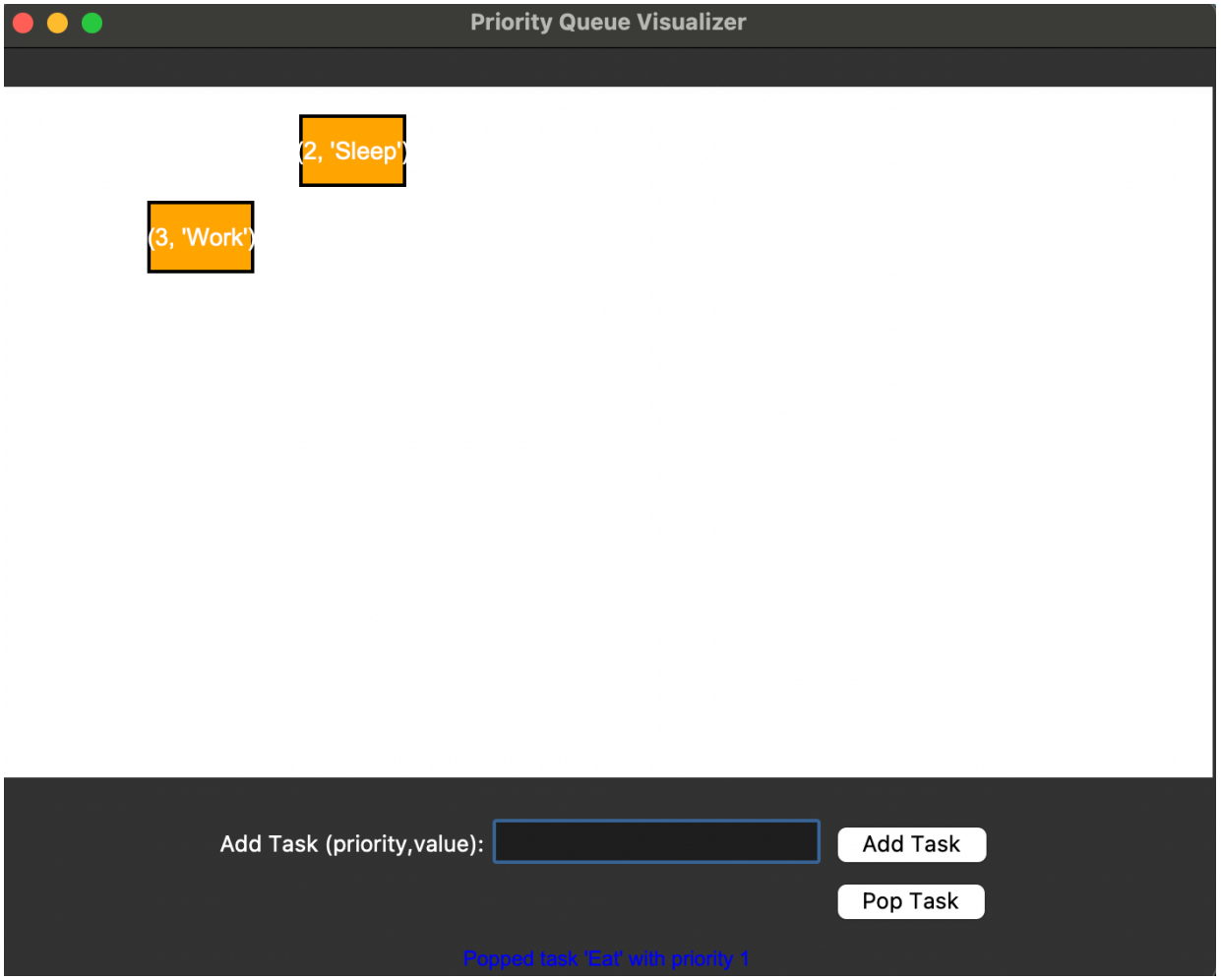
Test the visualiser for pushing a sequence of tasks with priority in terms of priority and value.

For example (1, Eat), (2,Sleep), (3, Work). Show the results (screenshots) of how the push and pop operations for a priority are performed in FIFO, but based on priority assigned.

- Test results when the **Add Task** button is used to insert tasks with priority value



- Test results when the **Pop Task** button retrieves value from the stack.



- Please complete this report with your python scripts and test results, and your comments for each task in relation to the questions asked.
- Demonstrate/present the tasks to your tutor in class.
- Prepare a report, similar to this handout, but with code/scripts and results included for your tests for each task, and save the report as **Week_6_lab_report.pdf**.
- Include this report as well as python scripts for each task (task 1 to task 4) in a separate folder and zip it, with a title (**Week_6_LabAcitivities.zip**).
- Upload the zip file to your github repo, and add a commit message with date and time details.
- You can leave the lab now. Please note your mark is 0 if you do not present/demonstrate your working project in lab class. This must be done in **Week 6**.

Mark for this tutorial: 8 marks